



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

SCHOOL OF COMPUTER SCIENCE AND STATISTICS

MODERN DELAY-TOLERANT EMAIL

RAKESH LAKSHMANAN

STEPHEN FARRELL

JULY 28, 2026

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
M.SC. COMPUTER SCIENCE - FUTURE NETWORKED SYSTEMS

Abstract

Abstract content

Acknowledgements

I would like to thank my supervisor, Professor Stephen Farrell. Under his supervision, I was able to successfully complete this dissertation. His inspiration and suggestions regarding the project's direction were essential for the progress of this research.

I am also grateful to my family and friends for their support and encouragement throughout this process.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Motivation	2
1.3 Research Question and Objectives	2
1.4 Dissertation Overview	4
2 Literature Review	5
3 Literature Review	6
3.1 Email over DTN	6
3.2 Anti-spam filtering	7
3.3 DTN security	8
3.4 Related problems	8
3.5 Methods and their limits	10
3.6 The gap	11
3.7 Summary	11
4 System Design	12
4.1 System Architecture	12
4.2 Environment and Technology Selection	14
4.2.1 Virtual machines over containers	14

4.3	Email stack	15
4.4	DTN stack	15
4.5	Anti-Spam Tool Selection	15
4.6	DTN Design	16
4.7	Delay Simulation Design	16
4.8	Summary	17
5	Implementation	18
5.1	Basis and Scope of This Work	18
5.2	Environment and Dependencies	18
5.3	DTN Layer	19
5.4	Mail Layer	20
5.4.1	Build	21
5.4.2	DTPC configuration	21
5.4.3	Postfix and Dovecot	22
5.5	Delay Simulation	22
5.6	DNS Resolver	23
5.7	Anti-Spam Tools	24
5.7.1	SpamAssassin	24
5.7.2	Rspamd	25
5.7.3	Vipul's Razor	26
5.8	Run Harness	26
5.8.1	Structure	27
5.8.2	Preflight and boot	27
5.8.3	Verification gates	27
5.8.4	Sequence of a run	28
5.8.5	Completion and collection	30
5.9	Running an Experiment	31
5.10	Summary	31

6	Results and Discussion	32
6.1	SpamAssassin	32
6.1.1	Detection	32
6.1.2	Scan time	34
6.1.3	Flips	34
6.1.4	Network volume	35
6.2	rspamd	35
6.2.1	Detection	36
6.2.2	Scan time	36
6.2.3	Flips, Run 2 to Run 3	37
6.2.4	Network volume	37
6.3	Vipul's Razor	38
6.3.1	Detection	38
6.3.2	Scan time	39
6.3.3	Flips	39
6.3.4	Network volume	40
7	Conclusions and Future Work	41

List of Figures

4.1	Testbed topology. Each client connects only to its local mail server. The relay is the only path between the Earth and Moon networks and is where the one-way link delay is applied.	13
4.2	End-to-end path of an email from Earth to Moon across the DTN link. .	14
4.3	Asymmetric delay. Traffic originating from the remote node is delayed on ingress through an ifb device, before NAT rewrites the source. Earth-origin traffic is not delayed.	17

List of Tables

6.1	Aggregate results across the three runs.	32
6.2	DNSBL/URIBL rule firing (selected rules); every rule falls from Run 2 to Run 3.	33
6.3	DNSBL-hit rate by corpus position; Run 3 collapses after $\approx$ email 500. . .	33
6.4	Scan-time distribution (emails per bucket).	34
6.5	Verdict changes and blocklist drift, Run 2 to Run 3.	34
6.6	Traffic on the relay: DNS to the resolver vs DTN bundle traffic.	35
6.7	Detection and scoring across the three runs.	36
6.8	Scan-time distribution across the three runs (messages per bucket). . . .	36
6.9	Per-message flips between the two delayed runs, which differ only in task_timeout.	37
6.10	Network volume per run. Capture durations are omitted: the captures ran unattended, so their wall-clock length does not reflect run time. . . .	37
6.11	Verdict distribution across the three runs.	38
6.12	Scan-time distribution across the three runs.	39
6.13	Per-message flips, matched on a Message-ID and Delivery-date key. Three Run 1 messages with short duplicate keys are excluded from the left table.	39
6.14	Network volume per run.	40

1 | Introduction

1.1 Background

Communication in space is not like communication on Earth. The distances are large, so signals take time to travel: a one-way trip between Earth and the Moon takes about 1.3 seconds, and to Mars it takes minutes. Links also break for long periods, as bodies move and block the line of sight. The normal internet assumes the opposite, short delays and a connection that stays up, so its protocols fail here: they expect a reply within a short time and treat a missing one as an error.

Delay-Tolerant Networking (DTN) was designed for these conditions (1). Instead of a live end-to-end connection, it uses store-and-forward: each node holds a message until the next link is available, then passes it on. The Bundle Protocol carries messages as self-contained units called bundles between DTN nodes (2). This tolerates long delays and broken links, which makes DTN the accepted basis for interplanetary networking (3), an active area with real implementations (4, 5).

Email is one of the basic tools people expect, so carrying it over DTN has received attention. A gateway translates between standard mail protocols and the Bundle Protocol (6, 7), and open tools implement this (8), so a message can move from a mail server on one side of a DTN link to a mail server on the other.

On any mail system, much of the incoming mail is spam, and anti-spam tools are a standard part of the setup. These tools do more than read the message text. They also test the sender and the links in the message against live network services: DNS

blocklists of known-bad addresses, sender reputation systems, and shared databases of known spam. Every one of these checks sends a request to a remote server during the scan and waits for a fast answer before it can score that rule.

1.2 Motivation

Work on DTN email has gone into moving mail across the link, not into filtering it once it arrives. Yet filtering is part of running a mail service, not an optional extra. Consider a crewed Moon base whose users exchange email with Earth over a DTN link (9): spam will reach it as it reaches any mail system, and the tools that would catch it now depend on lookups that must cross the delayed link.

Whether those tools still work is not obvious. A DTN link breaks the assumption they rely on: the answer is now seconds away, or the link is down when the request is made. A tool might slow to a crawl while it waits, lose detection when answers arrive too late, give up early and let spam through, retry until it loads the link, or fail with no sign that it failed. Each tool reaches the network in its own way, so each may respond differently. None of this has been measured.

This dissertation measures it. The value is in understanding how the tools behave under delay and why, which is a starting point for anyone who later builds spam filtering into interplanetary email.

1.3 Research Question and Objectives

This dissertation turns that gap into a precise question.

Research question. The primary research question is:

How do existing anti-spam tools behave when run over a delay-tolerant network link, and what mechanisms drive that behaviour?

It breaks into three sub-questions:

- How does each tool's network dependence interact with propagation delay?
- What failure modes appear under delay, and what causes them?
- How do the tools' own timeout and retry settings behave under delay, and does tuning them change the outcome?

Objectives. To answer this, the dissertation sets out to:

1. Build a controlled Earth–Moon DTN email testbed using real mail and DTN software, with a link delay that can be set to a chosen value.
2. Assemble a representative email corpus and a repeatable method to scan it and record per-email timing, verdicts, and network traffic.
3. Evaluate four tools, SpamAssassin, Rspamd, Vipul's Razor, and DCC, chosen to cover the main ways a filter depends on the network.
4. For each tool, compare its behaviour under delay against a no-delay baseline and identify the mechanisms behind the differences and any failure modes.
5. Draw the cross-tool comparison: what generalises about anti-spam filtering that depends on synchronous network services when it runs under delay.

Scope. This is a behavioural study: it characterises and explains the tools' response to delay, and does not propose or build a filtering system for interplanetary email.

Contributions. The dissertation makes three contributions:

- A reproducible Earth–Moon DTN email testbed for evaluating mail-layer tools under controlled delay.
- An empirical, mechanism-level account of how four anti-spam tools behave under delay, including the failure modes and their causes.
- A measurement method that separates sourced fact from interpretation,

controls for blacklist drift, and uses back-to-back runs so that delay effects can be isolated.

1.4 Dissertation Overview

This dissertation is organised into six chapters. Chapter 1 has introduced the problem, the research question, and the objectives. Chapter 2 reviews the background: delay-tolerant networking and the Bundle Protocol, the email system and how mail is carried over DTN, and the anti-spam tools and the network services they depend on. Chapter 3 describes the design of the testbed, including the node layout, the choice of software, and the delay simulation. Chapter 4 covers the implementation, from building the nodes to configuring the mail, DTN, and anti-spam software. Chapter 5 presents the evaluation: each of the four tools is measured under delay against a no-delay baseline, and the results are compared. Chapter 6 concludes and sets out directions for future work.

2 | Literature Review

3 | Literature Review

This chapter reviews the work that surrounds spam filtering over a DTN link. It covers three fields that touch the subject, a set of problems elsewhere that have the same shape, and the methods those fields use to evaluate their systems. It ends by setting out what none of them answers.

3.1 Email over DTN

Transporting emails over a delayed or intermittent network connection is not a novel thing. The previous studies are terrestrial and were developed for rural regions with no continuous connection. DakNet was using automobiles to transport emails from village to village, and to the internet gateway, thereby offering an inexpensive alternative for connectivity in contrast to a wired connection (10). KioskNet further improved on this by developing a whole system including kiosk controllers, naming, routing and security framework, reporting pilot installation (11). MotoPost employed dual mode mobile phones as data mules, and SMTP as the base application protocol for message transfer (12).

Security concerns are taken care of as identity and confidentiality issues for all three. KioskNet uses a public key infrastructure framework to provide security as privacy, confidentiality and integrity and takes care of user management and logging (11). None of the three has filters to eliminate junk mails, and none reports how many of those are received during their deployments.

The latest set of draft standards addresses space. One of them concerns a gateway that terminates the SMTP session locally and passes the delivered message to the Bundle Protocol (6). Another draft discusses how to encapsulate the message into a bundle (7). A third one concerns a naming model for out-of-this-world domains and the need to arrange the message authentication components, including DMARC, so that only local to the network resources are necessary to handle the request (13). BMail is an open implementation of the gateway approach (8).

Spam is mentioned in just one of those drafts, which states that spam can cause denial of service problems to a store-and-forward network and recommends two solutions for it: checking the integrity and authenticity of the sending network, as in the case of DMARC, and whitelisting the sending networks (6). These are preventive measures addressing the question of who is allowed to send messages. Neither mentions any tool that could examine messages actually arriving. The security section of the encapsulation draft merely notes that the payload with bad data is, per se, a denial of service (7).

3.2 Anti-spam filtering

The spam detection area is large and entirely focuses on classification accuracy. This survey on machine learning methods describes major techniques, their comparative advantages and open issues in the perspective of filter ability to separate spam from non-spam messages (14). The digest-based approach to detecting spam messages is also considered in such terms: the algorithm underlying collaborative filtering was tested in respect of three criteria: its robustness to automation, robustness to attacks and false positive rate (15).

The underlying mechanisms implemented by the tools used in this study are described by their projects rather than in academic papers. The way how blocklist is published and queried in the form of DNS zone is described by Spamhaus (16). The network configuration of SpamAssassin contains the description of its network rules

and parameters that constrain them (17). URI blocklist plugin of the program also contains the detailed description of its DNS query starting from resolving name servers to address (18). The sender authentication techniques SPF, DKIM and DMARC are specified in RFC 7208 (19), RFC 6376 (20) and RFC 7489 (21) correspondingly. Rspamd (22) and Vipul's Razor (23) are also documented by their projects.

What this body of work measures is detection rate and false positives. What it does not measure is time. The evaluations are run on a network where a lookup returns in milliseconds, so the cost of reaching a remote service never appears as a variable.

3.3 DTN security

The security architecture for DTN defines the security requirements of the store-and-forward network and rationale behind choosing or rejecting specific security measures (24). The problem addressed by the security architecture for DTN is scarcity of resources on the node and utilization of those resources by inappropriate traffic. The solution proposed by this architecture is authentication of the bundle to ensure that the bundle has originated from a legitimate entity and can use the network. In other words, there is a common assumption between the current work and this research, but different solutions are being provided.

3.4 Related problems

Four problems already studied elsewhere have the same structure as the one examined here.

The first is the round-trip costs embedded in the protocol, which will be transported over DTN. The process of routing that defined the DTN problem recognized the importance of the DNS query and response, as well as a reliable transport exchange

for an SMTP transaction (25). SMTP itself was solved by terminating the connection over a gateway (6). This issue arose once again with HTTP encapsulated over the Bundle Protocol; this draft explains that while version negotiation and URL redirection may still be possible in the proposed solution, they are not recommended, since they will require further bundling over the delayed network (26). The issue here is the same issue that is important to the network lookups performed by the filter.

The second issue is sequential DNS lookups. Spamhaus explains that performing blocklist queries sequentially rather than simultaneously increases the cost of the individual query from tens to hundreds of milliseconds, and aggregates in cost among a set of queries, mentioning that a normal mail transaction will have tens of these queries (27). This increase will be minimal on a terrestrial network, but the principle is significant under delay.

The third is that the latency of network tests is already recognised by people who run these tools. Advice circulated on the SpamAssassin users list, and since kept on the project wiki, distinguishes processor time from latency and states that the network tests, both blocklist and digest, add significant latency while costing little processor time; it suggests allowing fifteen to twenty seconds per message so that these tests do not time out (28). The figure is set for a network where a lookup takes milliseconds.

The fourth is where filtering runs. One study moves the filtering system from the receiving server to the sending server in order to reduce detection time (29), which treats the position of the filter as a design variable rather than a fixed property of a mail system.

The third one is that the latency of network tests has been well recognized by those who operate such testing methods. The tips posted in SpamAssassin users mailing list, and kept on the SpamAssassin project wiki as well, describe the difference between processor time and latency and say that the network tests, including

blocklist and digest ones, cause considerable latency but require insignificant processor time and advise to give up to fifteen or twenty seconds for each message not to timeout the test (28). This number is chosen for the network with millisecond lookups.

The fourth one is where the filtering process occurs. One paper relocates the filtering system from the receiving server to the sending server in order to reduce detection time (29).

3.5 Methods and their limits

Each of these fields evaluates in a way that could not produce the result this dissertation needs.

The DTN application evaluation reasons from the specification. The HTTP encapsulation draft identified the round-trip problem through protocol analysis and designed around it (26). This works when the round-trips can be seen in the protocol. The network checks made by the filters are not: they happen in the scanner process according to plugin behavior, timeouts, and chains of lookup dependencies which do not appear in any protocol document. For example, the chain of URI blocklist lookups is specified only in the plugin documentation (18). Protocol analysis can not find them.

The anti-spam evaluation measures the performance of a filter on a corpus using a fast network and reporting accuracy (14, 15). When done this way, the method will produce no timing information whatsoever, and could not, because the latency required for evaluation does not exist.

The DTN evaluation uses simulation and emulation, measuring delivery ratio, routing behavior, and buffer occupancy (25).

This dissertation therefore runs the real tools over a real DTN stack under controlled delay, and records per-message timing, verdicts and packet captures. Behaviour that

is neither written in a specification nor visible in a terrestrial accuracy benchmark can only be found by running the software and observing the network.

3.6 The gap

There are four bodies of literature that fall slightly short of the research question. Deployments and drafts for DTN email transfer send mail through a delayed channel but they don't filter any mail. Literature about anti-spam filters measure the accuracy of classification filters but never measure the filters behavior under a delayed network. The DTN security literature considers the cost of undesirable traffic but it addresses it using authentication and not classification filters. The adjacent application literature identifies the round-trip time problem but solves it proactively without measurement.

The gateway draft identifies spam as a threat to a store-and-forward network but stops at policy (6). None of the studies measure the behavior of the tools due to delay in their lookups. This is the research question answered in this dissertation.

3.7 Summary

The transport problem for email over DTN has been addressed, by deployments and by standards work. The filtering problem has not. The tools that would do the filtering depend on network services whose cost has only ever been measured on a fast network, and the fields that would otherwise study this use methods that cannot observe it. The next chapter describes the testbed built to measure it directly.

4 | System Design

This chapter describes the testbed used to evaluate anti-spam tools under the one-way delay of a deep-space DTN link. It sets out the node layout, the host model and the reasons for it, the tools under test, the DTN configuration, the delay simulation, and the experimental design.

4.1 System Architecture

The testbed has two networks joined by a relay. Each node is a separate virtual machine running Ubuntu 22.04 LTS. Earth uses the 192.168.100.0/24 subnet and Moon uses the 192.168.200.0/24 subnet. The nodes are:

- **Mail1 (Earth).** The Earth mail server. Runs Postfix, Dovecot, and the four anti-spam tools. Address 192.168.100.10. ION node 1.
- **Space (Relay).** The transit node between the two networks. Runs an ION node and carries the delay and packet-capture configuration. Earth-side interface 192.168.100.1, Moon-side interface 192.168.200.1. ION node 2.
- **Mail2 (Moon).** The Moon mail server. Runs Postfix and Dovecot. Address 192.168.200.11. ION node 3.
- **Client1 (Earth).** User terminal on the Earth network. Runs Thunderbird and connects to Mail1 over IMAP (port 143) and SMTP submission (port 587, STARTTLS).

- **Client2 (Moon).** User terminal on the Moon network. Runs Thunderbird and connects to Mail2 the same way.

Mail moves server to server across the DTN link: a message submitted on one mail server is carried as a bundle through the relay to the other. ION runs on all three servers, so the bundle path is node 1 (Earth) to node 2 (relay) to node 3 (Moon). The relay is the only path between the two networks, so all traffic between Earth and Moon passes through it. This is where the one-way link delay is applied, standing in for the Earth–Moon propagation time. Mailboxes use the mbox format, and each user reads mail from their local server. The topology is shown in Figure 4.1.

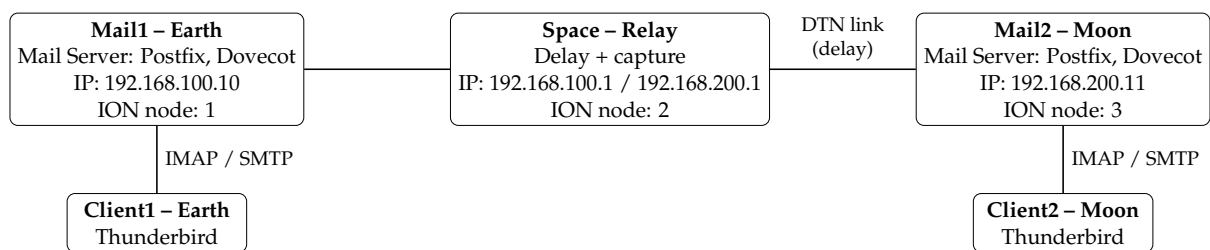


Figure 4.1: Testbed topology. Each client connects only to its local mail server. The relay is the only path between the Earth and Moon networks and is where the one-way link delay is applied.

Data flow (Earth to Moon). The path of a message is shown in Figure 4.2.

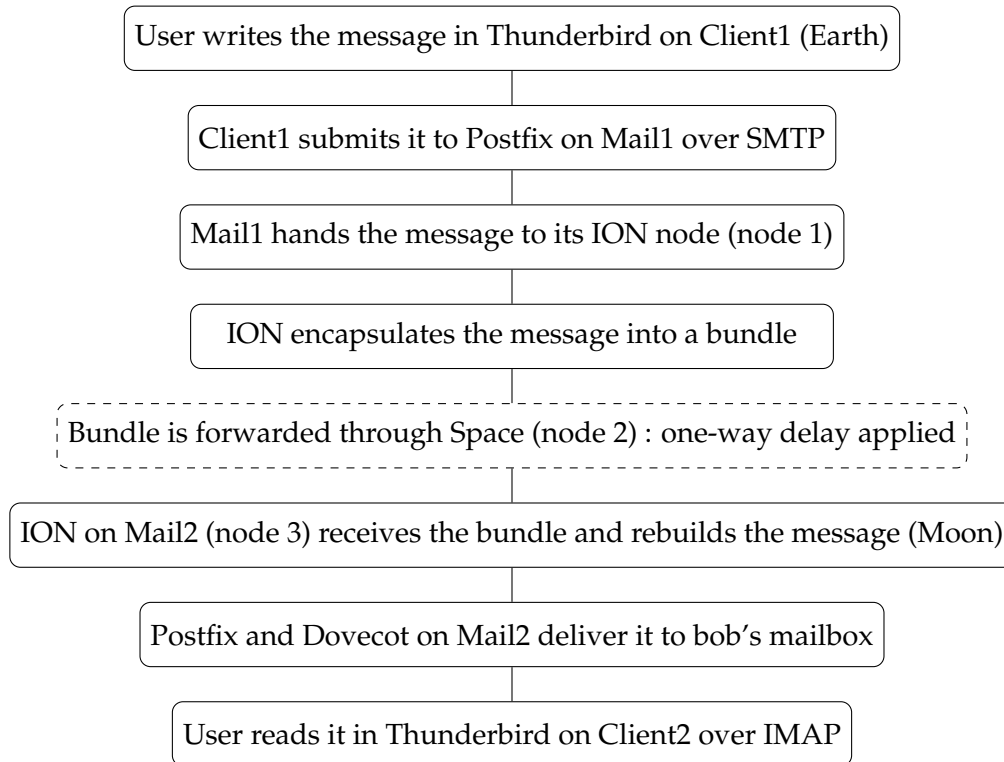


Figure 4.2: End-to-end path of an email from Earth to Moon across the DTN link.

4.2 Environment and Technology Selection

4.2.1 Virtual machines over containers

The testbed runs on full virtual machines (Ubuntu 22.04 LTS, ARM64, under UTM on macOS). Containers were considered but not used. The anti-spam experiment has three needs that decide this:

- **VMs match the real topology.** The testbed mimics three separate machines — Earth, Space, Moon — connected by a long-delay link, and full VMs naturally model this, whereas containers all share one kernel on one host.
- **Simpler delay setup** Setting up `ifb` to delay incoming traffic is fiddly and error-prone inside containers, but it's straightforward on a VM where each node has its own kernel and network stack.
- **DTN state stays isolated per node.** Each ION node keeps its own bundle store

and contact schedule, and a full VM ensures the link delay sits on the path itself rather than leaking into a node's own processing.

4.3 Email stack

- **Postfix** was chosen for its clear interface for handing mail to an external program, which is needed to pass messages to the DTN layer.
- **Dovecot** was chosen because it works directly with the mbox format and the local system accounts that Postfix delivers to.
- **Thunderbird** was chosen as a real mail client, so the message traffic in the testbed matches what a normal user would generate.

4.4 DTN stack

- **ION-DTN** was chosen for its deep-space flight heritage and its existing tool for carrying email over the Bundle Protocol, which connects directly to the mail stack.

4.5 Anti-Spam Tool Selection

Four tools were selected, each with a different external dependency:

- **SpamAssassin 3.4.6** — content scoring that depends on DNS blocklist lookups.
- **Rspamd 2.7** — content scoring with a large number of asynchronous DNS lookups.
- **Vipul's Razor 2.84** — collaborative filtering through a central nonce server, reached over TCP on port 2703.
- **DCC 2.3.169** — collaborative filtering through checksum servers, reached over UDP on port 6277.

The four cover the main ways a filter reaches the network: blocklist DNS (SpamAssassin, Rspamd), a TCP request to a central reputation server (Razor), and a UDP request to a checksum server (DCC). Testing one tool would show only one failure path.

4.6 DTN Design

The DTN layer uses ION-DTN 4.1.4 and BPMail. The three servers run ION as nodes on a single ipn scheme: Mail1 (Earth) is node 1, Space (relay) is node 2, and Mail2 (Moon) is node 3. Bundles are carried over a UDP convergence layer on port 4556, and the relay forwards them between the two mail nodes, so the path from Earth to Moon is node 1 to node 2 to node 3. The contact plan sets the windows during which the nodes can exchange bundles. BPMail sits above ION on each mail server: it takes a message from Postfix, encapsulates it into a bundle for the destination node, and rebuilds it on arrival. The full configuration, including the SDR settings, the contact plan, and the startup order, is given in the implementation chapter.

4.7 Delay Simulation Design

Delay is applied with netem. The key requirement is that delay is applied to one direction only — to traffic coming from the remote node — and not to the Earth node's own traffic.

A plain netem rule on the relay's interface delays everything that crosses it, including the Earth node's traffic, which would change what is being measured. To avoid this, ingress traffic from the remote side is redirected to an Intermediate Functional Block (ifb) device, and the delay is applied there. The redirect happens before the relay's NAT rewrites the source address, so the rule can match traffic by its true origin. Earth-origin traffic does not pass through this path and stays at normal latency. This is shown in Figure 4.3.

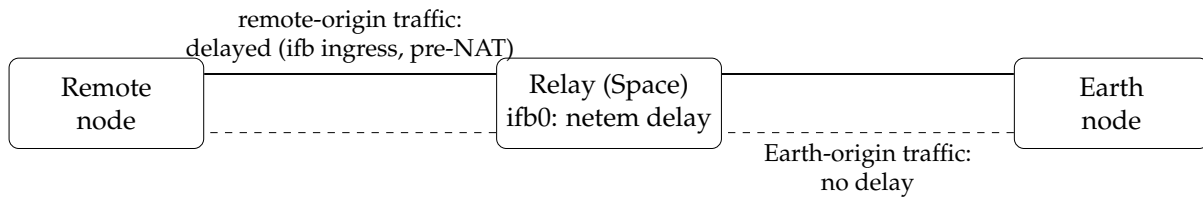


Figure 4.3: Asymmetric delay. Traffic originating from the remote node is delayed on ingress through an ifb device, before NAT rewrites the source. Earth-origin traffic is not delayed.

Delay is the independent variable. The experiments use a representative one-way delay of 1.3 seconds as a deep-space operating point. The testbed allows any value to be set, so the same setup covers the range from short Earth–Moon delays to multi-minute deep-space delays.

4.8 Summary

This chapter set out a three-node testbed for measuring anti-spam tool behaviour under link delay. The design uses full virtual machines for kernel and DNS isolation, a private resolver to give the Earth side a working DNS path, four tools chosen to cover the main external dependencies, and an ifb-based scheme to apply delay to one direction only. The next chapter describes how this design was built and configured, and the chapter after it presents the results.

5 | Implementation

This chapter describes how the testbed was built and how the experiments are run on it. It covers the DTN and mail layers, the delay mechanism, the resolver, the setup of each anti-spam tool, and the harness that executes a full set of runs without supervision.

5.1 Basis and Scope of This Work

The DTN transport layer of the testbed follows the approach established in earlier work on delay-tolerant email (?), which built and validated an ION and BPMail pipeline for carrying mail over the Bundle Protocol. The same components are used here, rebuilt on virtual machines rather than containers for the reasons given in the previous chapter, and are described only in the detail needed to make the results reproducible.

Everything above that layer is new: the delay mechanism on the relay, the external resolver, the installation and configuration of the three anti-spam tools, the measurement method, and the automation that runs the experiments and collects the results.

5.2 Environment and Dependencies

The testbed runs as five virtual machines on a single Apple Silicon MacBook Air under UTM. All five run Ubuntu 22.04 LTS (ARM64). Three are servers, mail1, space

and mail2; two are clients, one on each network. The servers carry the mail, DTN and anti-spam software. The clients run Thunderbird and are used only to confirm that the mail path works end to end; they take no part in the measurements.

Three libraries are newer than the versions in the Ubuntu 22.04 repositories and are compiled from source on each server before ION and BPMail are built: mbedtls 2.28.8, gmime 3.2.7 and c-ares 1.34.4. Each is built and installed in turn, followed by ldconfig so that the new shared libraries are found ahead of the system ones.

5.3 DTN Layer

ION-DTN 4.1.4 is built from source on all three servers:

```
./configure --disable-crypto CFLAGS="-Dlinux"  
make  
sudo make install  
sudo ldconfig
```

The `-Dlinux` flag is required because ION's `platform.h` selects the system headers that define `MAXPATHLEN` behind an `#ifdef linux` check. Crypto is disabled because the Bundle Protocol Security extensions are not used here.

Each node is configured from the `bench-udp` demo template supplied with ION. The node numbers are 1 for mail1, 2 for space and 3 for mail2. Bundles are carried over the UDP convergence layer, with each node listening on its own port: 1113 on mail1, 2113 on space and 3113 on mail2. The bundle protocol configuration for mail1 is:

```
a scheme ipn 'ipnfw' 'ipnadminep'  
a endpoint ipn:1.0 x  
a endpoint ipn:1.1 x  
a endpoint ipn:1.128 x  
a endpoint ipn:1.129 x
```

```
a protocol udp 1400 100
a induct udp 192.168.100.10:1113 udpcli
a outduct udp 192.168.100.1:2113 udpclo
```

Space is the only node with two inducts and two outducts, and is the only route between the two networks. Routing uses Contact Graph Routing, so routes are computed from the contact plan rather than configured statically. The contact plan is identical on all three nodes and declares reachability in both directions between each adjacent pair:

```
m horizon +0
a range +0 +86400 1 2 1
a range +0 +86400 2 1 1
a range +0 +86400 2 3 1
a range +0 +86400 3 2 1
a contact +0 +86400 1 2 100000000
a contact +0 +86400 2 1 100000000
a contact +0 +86400 2 3 100000000
a contact +0 +86400 3 2 100000000
```

The contact windows are set to 24 hours. Shorter windows expire during a run and bundles stop being forwarded, so the window has to cover a full pass of three runs with margin. Each node is started with an `ionstart` script that loads the node configuration, the contact plan, the security database and the bundle protocol configuration in that order, with short pauses between them.

5.4 Mail Layer

BPMail 0.1.0 provides the gateway between the mail system and ION. It is installed on mail1 and mail2 only; space forwards bundles and does not handle mail.

5.4.1 Build

BPMail is built with Meson. One change to the build configuration is required. The first build failed with an error reporting that no value was defined for MAXPATHLEN. The cause is an interaction between two settings. ION's `platform.h` includes the system headers that define MAXPATHLEN only when the `linux` platform macro is set, guarded by `#ifdef linux`. BPMail's `meson.build` specified `c_std=c17`, and strict ISO C17 disables non-standard platform macros such as `linux`, so the guard never passed and the definition was never picked up, even when `-Dlinux` was passed on the command line. Changing the standard to GNU C17 keeps those platform macros while remaining C17-compliant:

```
sed -i "s/c_std=c17/c_std=gnu17/" ~/bpmail/meson.build
```

The platform flag is supplied through a native file, and the build then completes:

```
printf '[built-in options]\nc_args = [\x27-Dlinux\x27]\n' > linux-native.txt
meson setup build --native-file linux-native.txt
ninja -C build
sudo ninja -C build install
```

5.4.2 DTPC configuration

BPMail transfers mail over DTPC rather than raw bundles, so the two mail nodes need DTPC endpoints and a profile. Service 128 is used for sending and 129 for receiving, and the endpoints are added to each node's bundle protocol configuration as shown above. The DTPC profile is the same on both nodes:

```
1
w 1
a profile 1 5 1000 10 600 0.1 dtn:none
s
```

The profile sets a maximum of five retransmissions, an aggregation size limit of 1000 bytes, an aggregation time limit of 10 seconds and a bundle lifetime of 600 seconds. The `dtpcadmin` step is appended to the `ionstart` script on both mail nodes so that DTPC comes up with the rest of ION.

A DTPC topic can only be opened once per node, for sending or receiving but not both, so each direction uses its own topic. The experiments send from mail1 to mail2, and use topic 26 for that direction. The destination endpoint is the receiving service on node 3, `ipn:3.129`; sending to the `service-128` endpoint does not reach the receiver.

5.4.3 Postfix and Dovecot

Postfix is configured on both mail nodes with a transport map that routes mail for the remote domain to a pipe transport, which hands the message to BPMail. BPMail needs access to ION's shared memory, which depends on the working directory, so the pipe calls a small wrapper that changes to the ION node directory first:

```
bpmail unix - n n - 1 pipe
    flags=F user=<user> argv=/home/<user>/bpmailsend_wrapper.sh 1 ipn:3.129
```

A background daemon runs `bpmailrecv` in a loop on each node and pipes what it receives into `sendmail`, which delivers it to the local mailbox. Dovecot serves those mailboxes over IMAP, in mbox format.

This path is what the Thunderbird clients use, and it is how the testbed was verified as a working mail system. It is not the path used for measurement. The experiments call `bpmailsend` and `bpmailrecv` directly, for the reasons given in Section 5.8.4.

5.5 Delay Simulation

The link delay is applied on space, using `tc` with the `netem` queueing discipline, attached to the Moon-facing interface. It therefore acts on all traffic crossing the

boundary between the two networks.

tc can only shape traffic leaving an interface. The outbound direction is straightforward: a netem qdisc is attached to the root of the Moon-facing interface. The inbound direction has no equivalent, because there is no egress queue to attach a qdisc to. The usual way around this is an Intermediate Functional Block (ifb) device, a virtual interface whose own egress path can be shaped normally. Traffic arriving on the Moon-facing interface is matched by a filter and redirected to the ifb device with a mirrored action, and a second netem qdisc on the ifb device applies the same delay. The full sequence is:

```
sudo modprobe ifb
sudo ip link add ifb0 type ifb
sudo ip link set ifb0 up
sudo tc qdisc add dev enp0s2 root netem delay 1300ms
sudo tc qdisc add dev enp0s2 handle ffff: ingress
sudo tc filter add dev enp0s2 parent ffff: protocol ip u32 match u32 0 0 \
    action mirrored egress redirect dev ifb0
sudo tc qdisc add dev ifb0 root netem delay 1300ms
```

Clearing the delay removes both qdiscs, the ingress filter and the ifb device.

The one-way delay is a parameter. The experiments use 1300 ms in each direction, giving a round-trip time of about 2.6 s. Because the delay sits at the boundary of the Moon network, everything mail2 exchanges with anything outside it pays that cost: the bundles it receives from mail1, and the DNS queries the scanner makes while examining a message.

5.6 DNS Resolver

The scanners run on mail2 and need working DNS. The queries are served by a local unbound instance on mail2, which forwards to a private recursive resolver running on

an Azure virtual machine on port 5353. The reasons for using a private resolver are given in the previous chapter.

Two changes to the default unbound configuration were needed.

The first was a startup failure. With DNSSEC validation enabled and all queries forwarded, unbound logged repeated failures to prime the trust anchor and refused to resolve anything. Forwarding and validation conflict in this configuration. Because the upstream resolver performs validation, the validator module can be removed on mail2, leaving only the iterator:

```
module-config: "iterator"
```

The second concerns two timers that interact with the link delay:

```
discard-timeout: 9000
infra-cache-min-rtt: 4000
```

Both defaults are shorter than the round-trip time of the delayed link. The reason these values are needed, and what happens with the defaults in place, is a result of the experiments and is presented in Chapter 5.

5.7 Anti-Spam Tools

Three tools are installed on mail2, the node that receives and scans. Each is invoked once per message, on the message file, and each is run under two configurations: a default one, and one with its timeout reduced below the round-trip time of the link.

5.7.1 SpamAssassin

SpamAssassin is invoked through its standalone binary rather than through

spamc:

```
spamassassin < "$FILE"
```


The client and daemon pair was tried first and had to be abandoned. When `spamc` cannot reach `spamd` it fails open, passing the message through unchanged, so every message came back with no `X-Spam-Status` header and no indication that anything had gone wrong. The standalone binary scans without a daemon, and re-reads `local.cf` on every invocation, which means a configuration change takes effect on the next message with no restart.

The two configurations differ only in whether the timeouts are set. The default profile removes any timeout lines from `local.cf`, leaving SpamAssassin's own defaults of 15 s for `rbl_timeout` and 5 s for `spf_timeout`. The reduced profile sets both to 1.5 s, below the 2.6 s round-trip time:

```
sudo sed -i '/^rbl_timeout /d;/^spf_timeout /d' /etc/spamassassin/local.cf
echo 'rbl_timeout 1.5' | sudo tee -a /etc/spamassassin/local.cf
echo 'spf_timeout 1.5' | sudo tee -a /etc/spamassassin/local.cf
```

5.7.2 Rspamd

Rspamd runs as a daemon and is scanned through its client:

```
rspamc --ip 1.2.3.4 --mime < "$FILE"
```

The `-ip` option supplies a client address so that the message is treated as having arrived from outside. Without it, mail presented locally is treated as internal and the network checks that this study is concerned with are skipped.

Rspamd caches state in the running daemon, so it is restarted before each run to start from a cold cache. The reduced-timeout profile writes a single override file and restarts:

```
echo 'task_timeout = 1.5s;' | sudo tee /etc/rspamd/local.d/options_override.inc
sudo systemctl restart rspamd
```

The default profile removes that file and restarts.

5.7.3 Vipul's Razor

Razor is invoked with `razor-check`, which reports its verdict through the exit code: 0 if the message is catalogued as spam, 1 if it is not, and 2 on a network error. The third case matters here, because it separates a decision from a failure to reach the server:

```
razor-check < "$FILE"
```

Razor's server list is refreshed with `razor-admin -discover` before each run.

Razor differs from the other two in one respect that affects the experiment directly: it has no timeout setting in `razor-agent.conf`. Its connection timeouts are fixed in the client source. Producing the reduced-timeout condition therefore requires patching the module that holds them, keeping a copy of the original so the default condition can be restored:

```
sudo cp -n "$CORE_PM" "$CORE_PM.orig"
sudo sed -i 's/Timeout *=> *20/Timeout => 2/g' "$CORE_PM"
```

That a source patch is the only way to change this value is itself a finding about the tool, and is returned to in Chapter 5.

5.8 Run Harness

The three runs for one tool take several hours and involve three machines, so the experiments are automated. The harness is a Python program that runs on the Mac and drives the testbed over SSH. A single command performs a complete pass:

```
python3 run.py --tool spamassassin
```

Every parameter it uses is held in one configuration file, so the procedure is identical between tools and between passes, and the file itself records the exact settings a set of results was produced under.

5.8.1 Structure

The harness connects to the three servers with Paramiko. Mail1 and mail2 are not reachable directly from the Mac, so their connections are tunnelled through space, which is opened first.

Work on the servers is done by two shell agents, one on mail1 that sends and one on mail2 that receives and scans. They are written out to each node at the start of a run, together with an environment file holding that run's settings, and launched detached so that they survive the SSH session. Each agent reports through two files in its working directory: STATUS, which holds RUNNING, DONE or FAILED, and progress, which holds a count of messages handled. The host polls these every thirty seconds and prints a line for each. The alternative, holding a live channel open to each node for the length of a run, is fragile over hours; sentinel files survive a dropped connection.

5.8.2 Preflight and boot

Before anything starts, the harness checks that `utmctl` is usable, that the named virtual machines exist, and that the results directory has at least 20 GB free. It then starts the machines in order and waits for each to accept SSH, up to five minutes. It also holds `caffeinate` open for the session so the Mac does not sleep partway through a run.

5.8.3 Verification gates

A run only starts if the testbed is in a known state. Four checks are made, and any of them can abort the pass:

- **Clocks.** Timing is compared between the sender on mail1 and the receiver on mail2, so their clocks must agree. The offset from the host is measured on each node; anything above one second aborts.

- **Resolver.** unbound must be active on mail2 and must answer a test query.
- **ION.** ION is restarted across all three nodes, which resets the contact window so that it covers the whole pass.
- **Tool.** The tool's version command must return something, which confirms it is installed; the version string is recorded with the results.

5.8.4 Sequence of a run

Each run follows the same sequence.

The testbed is first returned to a cold state: unbound is restarted, which clears both its cache and its record of upstream server behaviour, any agent state from a previous run is deleted, the tool is reset, and any leftover netem configuration is removed. The resolver's infrastructure cache is dumped and saved, so that its state before the run is on record.

The tool profile for the run is then applied, and the delay if the run calls for it. The round-trip time is then measured by pinging mail1 from mail2, and checked against a window: 2400 to 2900 ms for a delayed run, and under 150 ms for the baseline. A measurement outside the window aborts the run. This is the most important gate in the harness. A run whose delay was not actually in place, or was applied twice, would produce plausible-looking numbers that mean nothing, and the failure would not be obvious afterwards.

Packet capture is then started on space and on mail2, excluding management SSH so that the harness's own traffic does not appear in the bandwidth figures:

```
sudo tcpdump -i any -s 0 -w {out} -C 100 -W 30 'not tcp port 22'
```

The receiver is launched first and the sender three seconds later, so that no message can arrive before the receiver is listening.

The sender walks the corpus in sorted order. For each message it records the

Message-ID and a timestamp, sends it, and records the exit code:

```
mid=$(grep -i -m1 '^Message-ID:' "$f" | tr -d '\r\n' | cut -d' ' -f2-)
sent=$(date -u +%s.%N)
eval "$SEND_CMD"
rc=$?
echo "$seq,$bn,$mid,$sent,$rc" >> "$SENT"
```

The send command pipes the message directly to BPMail:

```
sed "1{/^From /d}" "$FILE" | bpsend -t 26 1 ipn:3.129
```

The leading From line of the mbox-format corpus file is stripped, because it is not part of the message.

The receiver blocks on bprecv until a bundle arrives, writes the message out, scans it, and records the timing:

```
RAW=$(timeout "$RCV_TIMEOUT" bash -c "$RCV_CMD")
arrived=$(date -u +%s.%N)
printf '%s' "$RAW" > "$eml"
start=$(date -u +%s.%N)
eval "$SCAN_CMD" > "$outf" 2>&1
rc=$?
end=$(date -u +%s.%N)
```

The scan output is saved in full for every message, not just the parsed verdict, so that the parsing can be revised later without repeating a run.

Sending and receiving through BPMail directly, rather than through Postfix, gives a clean boundary for the scan time. The clock starts when the message arrives and stops when the tool exits, so the recorded time is the time the tool spent on that message. Invoking the tool through Postfix would include its queueing in every measurement, and at a round-trip time of 2.6 s the two could not be separated.

Messages are paced rather than sent back to back. The interval differs by run and is set in the configuration.

5.8.5 Completion and collection

When the sender reports DONE the harness enters a drain phase, because messages are still in flight and will keep arriving for at least one one-way delay afterwards. It waits for the receiver's count to reach the number sent, up to fifteen minutes, then writes a STOP file that the receiver checks between bundles so that it exits cleanly rather than being killed mid-scan.

Captures are stopped, and the resolver's state is recorded again. Two health figures are taken from it: the highest retransmit timeout in the infrastructure cache, and the number of log entries reporting answers discarded for arriving too late. A run whose highest retransmit timeout has reached 120,000 ms is marked SUSPECT in its manifest, because that is the ceiling and indicates the resolver stopped working properly during the run. This is checked automatically so that a compromised run is flagged at the time rather than discovered later.

All artifacts are then pulled back to the Mac. The sent and received records are reconciled by Message-ID to identify any messages that did not arrive, and the receiver's timing records are joined with the parsed scan output into a single per-message file. Each tool has its own parser, because the three report their verdicts differently: SpamAssassin in an X-Spam-Status header that folds across lines, Rspamd in X-Spam-* headers added to the message, and Razor in its exit code alone.

Each run finally writes a manifest recording the tool and version, the profile, the measured round-trip time and the gate it was checked against, the timings, the message counts, the resolver health figures and the resulting status. The manifest is what makes a set of results interpretable months later.

5.9 Running an Experiment

A pass consists of three runs against the same corpus of 681 messages:

- **Run 1**, no delay, default tool configuration. The baseline.
- **Run 2**, delay applied, default tool configuration.
- **Run 3**, delay applied, tool timeout reduced below the round-trip time.

Runs 2 and 3 are executed in the same pass, one after the other, so that the two delayed conditions are separated by as little time as possible. External blocklist state changes over hours, and comparing two runs made far apart measures that change as much as it measures the effect of the configuration. The reasoning behind this is developed with the results in Chapter 5.

A completed pass leaves, for each run, the sender's record, the receiver's timing record, every scan output, packet captures from space and mail2, the resolver state before and after, the joined per-message file, and the manifest. A summary across the three runs is written at the end of the pass.

5.10 Summary

The testbed is a working DTN mail system with a controllable link delay, a resolver that reaches real blocklists, and three anti-spam tools each configurable into a default and a reduced-timeout condition. The experiments are driven by a configuration-file-based harness that verifies the testbed before each run, refuses to proceed if the delay is not what it should be, collects every artifact, and flags runs whose resolver state indicates the results cannot be trusted. The next chapter presents what those runs measured.

6 | Results and Discussion

6.1 SpamAssassin

SpamAssassin was run three times over the 681-email corpus: a no-delay baseline, Run 2 (delay, default timeouts), and Run 3 (delay, reduced timeouts). Each message was scanned once by the standalone binary and timed from arrival to exit.

Metric	Baseline	Run 2	Run 3
Flagged YES	383 (56.2%)	355 (52.1%)	316 (46.4%)
Emails with DNSBL hit	416	400	251
Scan min/avg/max (s)	0.45/1.68/7.58	3.17/7.36/16.50	0.43/4.05/14.83

Table 6.1: Aggregate results across the three runs.

6.1.1 Detection

With no delay, 383 messages were flagged. Applying 1.3 s each way then broke detection at the resolver: DNSBL answers returned at about 2.64 s (one RTT). But, they were discarded by unbound before reaching the scanner with the error message drop reply, it is older than discard-timeout. The default value of discard-timeout, 1900 ms, is lower than the RTT of 2600 ms, thus all delayed responses were too old and discarded (30). Setting discard-timeout to 9000 and infra-cache-min-rtt to 4000, which were higher than the RTT, solved the problem, and a cold resolution took 2.694 s while the test message regained its score. DNS

With the resolver fixed, Run 2 nearly preserved detection (416 → 400 DNSBL emails) at about four times the scan time. Run 3 cut rbl_timeout and spf_timeout to 1.5 s,

below the RTT, trading detection for speed. Two things follow. First, every DNSBL rule fired less than in Run 2 and none fired more (Table 6.2).

Rule	Baseline	Run 2	Run 3
RCVD_IN_PBL	151	149	73
RCVD_IN_SBL_CSS	148	136	43
URIBL_DBL_SPAM	118	117	73
RCVD_IN_PSBL	108	93	67
RCVD_IN_MSPIKE_BL	103	93	32
RCVD_IN_XBL	82	75	20
RCVD_IN_BL_SPAMCOP_NET	75	63	12

Table 6.2: DNSBL/URIBL rule firing (selected rules); every rule falls from Run 2 to Run 3.

Second, the resolver failed around email 500. With the 1.5 s timeout, the scanner drops each query, but the reply does reach unbound at around 2.6 s; thus, from unbound’s perspective, its upstream timed out, so it doubles the retransmit timeout on each event. After around 500 emails, the rto reached its ceiling of 120,000 ms, on this build unbound-checkconf rejects `infra-cache-max-rtt`, so nothing lowers that ceiling, and it did not recover until the resolver was restarted. The hit rate collapses in the tail (Table 6.3), so Run 3’s aggregate mixes a live-DNS portion with a dead-DNS tail, and timeout-effect comparisons must restrict to the live portion.

Corpus position	Run 2	Run 3
Emails 1–500	260/500 (52%)	220/500 (44%)
Emails 501–681	140/181 (77%)	31/181 (17%)

Table 6.3: DNSBL-hit rate by corpus position; Run 3 collapses after $\approx$ email 500.

The reason why the number of DNSBL hits in the case of Run 3 was 251 despite the timeout of 1.5 s that was below the RTT remains unexplained. Messages that were checked for a period longer than RTT had significantly more hits than messages scanned for a period shorter than the RTT.

6.1.2 Scan time

Even in baseline conditions, the scan is not instantaneous, the network rules depend on the DNSBL replies, which can take time, and the scan concludes only when the last one of them arrives; with the resolver set up properly, delayed scans were still taking around 8 seconds. Since the lookups go in serial waves, dependent on the results from the previous waves, this suggests that the latency depends on the depth of the dependency chain, not the number of lookups. There are two causes of such depth – URIBL chain, which resolves the name servers of the domain into IPs and performs DNSBL lookup of those IPs (domain > NS > IP > DNSBL), and nested SPF include: directives, each fetched before its children are visible.

Run	<1 s	1–3 s	3–6 s	6–10 s	>10 s
Baseline	438	101	104	38	0
Run 2	0	0	316	213	152
Run 3	162	129	194	184	12

Table 6.4: Scan-time distribution (emails per bucket).

The distribution changes completely with delay, since no Run 2 message scan completes in less than 3 s, since all must now do at least one round-trip time and dependency-chain messages must do many. Run 3 splits into two groups: the shorter timeout cuts many scans short, while dependency-chain messages still run long."

6.1.3 Flips

	Run 2 → Run 3
Verdict unchanged	424
Flipped YES → NO	148
Flipped NO → YES	109
Net flagged change	−39
Gained a DNSBL rule (drift)	178 (26.1%)

Table 6.5: Verdict changes and blocklist drift, Run 2 to Run 3.

Run 3 was done two days later than Run 2, so the effect for this pair is a combination of both. The shortened timeout aborts the DNS query before the answer comes back, eliminating DNSBL matches and shifting messages from YES to NO. However, 178 messages (26.1%) *acquired* the DNSBL rule between the runs, and a shortened timeout cannot do this, so this is blocklist drift over the two days. The 109 NO → YES switches are thus drift, while the 148 YES → NO switches are a mixture of timeout failure and drift.

6.1.4 Network volume

Component	Baseline	Run 2	Run 3
DNS bytes	5,160,681	5,418,742	3,413,349
DNS packets	37,789	39,875	25,783
Bundle bytes	572,625	538,529	547,709
Bundle packets	1,447	1,369	1,389
DNS % of bytes	90.0%	91.0%	86.2%

Table 6.6: Traffic on the relay: DNS to the resolver vs DTN bundle traffic.

Traffic in DNS is nine or ten times as much as the bundle bytes in the baseline and Run 2, while the number of bundle bytes is essentially the same between runs. The number of bytes for DNS traffic in Run 3 is lower since there is no longer any tail that sends out queries upstream; the bandwidth decrease is due to the collapse of the resolver tail.

6.2 rspamd

This section reports the three runs with rspamd. The runs follow the same corpus and DNS path used by the previous tool. rspamd was executed on the receiver using `rspamc -ip 1.2.3.4 -mime`; the `-ip` option ensures that the external processing is done, and the rules dependent on DNS run. Runs 1 and 2 were executed using the default task timeout, which was 8 s, while Run 3 uses 1.5 s, below the round-trip time.

6.2.1 Detection

Metric	Run 1 (baseline)	Run 2 (delay)	Run 3 (delay, 1.5 s)
Messages processed	681	681	681
Flagged as spam	675 (99.1%)	654 (96.0%)	569 (83.6%)
Messages with real hit	502	482	94
Messages with _FAIL	0	45	665
Total real hits	1449	1330	221
Total _FAIL	0	357	2729
Average score	18.13	17.20	9.26
Maximum score	46.10	45.55	21.55

Table 6.7: Detection and scoring across the three runs.

Baseline flags 675 of 681 messages (99.1%); this is the detection ceiling. Under delay with default timeouts (Run 2) detection is nearly preserved at 654 (96.0%). Reducing `task_timeout` to 1.5 s (Run 3) drops it to 569 (83.6%). The cause is in the hit and _FAIL rows: real hits fall 502 → 482 → 94 as timed-out lookups rise 0 → 45 → 665.

6.2.2 Scan time

Scan time (s)	Run 1	Run 2	Run 3
<1 s	644	0	2
1–2 s	24	0	679
2–4 s	1	113	0
4–6 s	6	428	0
≥6 s	6	140	0

Table 6.8: Scan-time distribution across the three runs (messages per bucket).

Baseline scans fast: 644 out of 681 scans in less than 1 s. In the delay setup with default timeouts (Run 2) every scan is slow (median 5.67 s), where the round trip time dominates since every uncached lookup incurs one RTT cost; 140 out of 681 scans hit the ceiling at 8 s. In Run 3, scan time reduced to 1 to 2 s for 679 out of 681 scans, with none above 1.64 s.

6.2.3 Flips, Run 2 to Run 3

	Run 2 → Run 3
Verdict unchanged	560
Flipped YES → NO	103
Flipped NO → YES	18
Net flagged change	−85

Table 6.9: Per-message flips between the two delayed runs, which differ only in `task_timeout`.

Reducing `task_timeout` flipped 103 messages from spam to not-spam and lowered the score of 579 of 681. The 18 messages that gained detection run counter to a shorter timeout and are most likely blocklist-listing drift between the two runs rather than a timeout effect.

6.2.4 Network volume

Metric (receiver capture)	Run 1	Run 2	Run 3
DNS queries sent	38,873	80,890	24,116
DNS responses received	38,771	80,822	24,078
Queries to upstream	13,894	19,033	5,969
Total packets	89 k	182 k	59 k
Total data	18 MB	32 MB	14 MB

Table 6.10: Network volume per run. Capture durations are omitted: the captures ran unattended, so their wall-clock length does not reflect run time.

The same number of sent and received queries is seen for all runs, meaning that there were no lost queries on the wire, while changes in traffic volume are due to `rspamd` itself, not to packet loss.

Run 2 sent almost twice the number of queries than the baseline ($38,873 \rightarrow 80,890$), and there was a proportional increase in data size ($18 \rightarrow 32$ MB). The reason is retransmission. `rspamd` retries sending the DNS request every time it times out waiting for the response, with the number of retransmits equal to the retransmit count, which gives a total per-request timeout of $\text{timeout} \times \text{retransmits}$, which is 1 s

· 5 by default. As the timeout per attempt is shorter than the 2.6 s round trip time, the unanswered queries will be resent until they get their response. So, an entirely fresh lookup will be sent 3 times on average until the answer is received at 2.6 s.

Run 3 does the opposite: query volume falls below than the baseline level (80,890 $\rightarrow$ 24,116 queries, 32 $\rightarrow$ 14 MB). Since the timeout is only 1.5 seconds, the task is terminated before the DNS budget expires (it is 5 seconds). Therefore, a request can be issued only twice before termination, stopping the sequence of retransmissions that inflated Run 2. This lower volume is the network-side view of the 2,729 _FAIL symbols: the queries went out but were abandoned before their answers returned.

6.3 Vipul's Razor

This section reports the three runs with Razor. The runs follow the same corpus and DNS path used by the previous tools. Razor 2.84 was executed on the receiver using `razor-check < message.eml`. Razor exposes no network timeout through `razor-agent.conf`, so the connect timeout is a hardcoded constant in `Razor2::Client::Core`. Runs 1 and 2 used the default of 20 s, while Run 3 patched it to 2 s, below the round-trip time.

6.3.1 Detection

Metric	Run 1 (baseline)	Run 2 (delay)	Run 3 (delay, patched)
Messages processed	681	681	681
YES (exit 0, spam)	22 (3.23%)	19 (2.79%)	0 (0%)
NO (exit 1, not spam)	659 (96.77%)	656 (96.33%)	0 (0%)
FAIL (exit 2, error)	0 (0%)	6 (0.88%)	681 (100%)

Table 6.11: Verdict distribution across the three runs.

Under delay with default timeouts (Run 2), the verdict remains essentially the same (3.23% $\rightarrow$ 2.79% flagged). Run 3 collapses the entire detection process as it reports

exit 2 in every scan performed. In contrast to the DNSBL-based tools, Razor has only one cooperative detection method, which means that if the connection fails, it reports no verdict at all.

6.3.2 Scan time

Scan time (s)	Run 1	Run 2	Run 3
<i>distribution (messages)</i>			
<1 s	673	0	0
1–5 s	4	0	0
5–10 s	0	90	671
10–15 s	0	465	10
15–30 s	1	73	0
30–60 s	0	44	0
≥60 s	0	9	0

Table 6.12: Scan-time distribution across the three runs.

Baseline scans are completed within one second (673 of 681). With delay without timeout (Run 2), the median increases to 11 s, and the distribution gets a heavy tail up to 122.6 s. With the timeout (Run 3), the time of scanning becomes deterministic with a value close to 8 s, which means four connection attempts of the server, each taking 2 s.

6.3.3 Flips

Transition (Run 1 → Run 2)	Messages	Transition (Run 2 → Run 3)	Messages
NO → NO (unchanged)	652	NO → FAIL	656
YES → YES (unchanged)	19	YES → FAIL	19
NO → FAIL	6	FAIL → FAIL	6
YES → NO	1		

Table 6.13: Per-message flips, matched on a Message-ID and Delivery-date key. Three Run 1 messages with short duplicate keys are excluded from the left table.

Under delay alone the verdict is stable: 671 of 678 comparable messages keep their baseline verdict. The timeout run flips 675 of 681 from a valid verdict to FAIL.

6.3.4 Network volume

Metric (receiver capture)	Run 1	Run 2	Run 3
Frames captured	12,814	25,997	57,937
Bytes captured	3.4 MB	5.1 MB	40.2 MB
TCP connections to :2703	687	887	5,460
Connections per message	1.01	1.30	8.02
DNS queries / responses	36 / 42	650 / 810	777 / 971

Table 6.14: Network volume per run.

DNS traffic is less than 1% of volume in all runs: Razor’s per-check DNS traffic is small, and its delay sensitivity is in TCP, not DNS. That is the opposite of rspamd, where DNS traffic dominated the delayed run.

Connections per message increase from $1.01 \rightarrow 1.30 \rightarrow 8.02$. Baseline uses about one server per message; with delay, some scans use additional servers due to failure; with the timeout, all the servers listed in the catalogue and nominations are used per-message because of failures of every handshake and the exhaustion of lists by `nextserver`. Therefore, the volume of Run 3 (57,937 frames, 40.2 MB) is the largest despite the shortest wall-time: every handshake failure has a kernel-level SYN retransmission, which accounts for roughly two SYNs per attempt across six servers in 681 messages.

7 | Conclusions and Future Work

Conclusions

Bibliography

- [1] Leigh Torgerson, Scott C. Burleigh, Howard Weiss, Adrian J. Hooke, Kevin Fall, Dr. Vinton G. Cerf, Keith Scott, and Robert C. Durst. Delay-Tolerant Networking Architecture. RFC 4838, April 2007. URL <https://www.rfc-editor.org/info/rfc4838>.
- [2] Scott Burleigh, Kevin Fall, and Edward J. Birrane. Bundle Protocol Version 7. RFC 9171, January 2022. URL <https://www.rfc-editor.org/info/rfc9171>.
- [3] Yosuke Kaneko, Vinton Cerf, Scott Pace, Jim Green, Laura DeNardis, James Schier, Dave J. Israel, Felix Flentge, Leigh Torgerson, Scott Burleigh, Marc Blanchet, Ed Birrane, Keith Scott, Oscar Garcia, Kiyohisa Suzuki, Marius Feldmann, Laura Chappell, Ginny Spicer, Henry Danielson, and Helen Tabunshchyk. Report: Solar System Internet Architecture and Governance. Technical report, Interplanetary Chapter, Internet Society, September 2023.
- [4] Scott Burleigh. Interplanetary overlay network: An implementation of the dtn bundle protocol. In *2007 4th IEEE Consumer Communications and Networking Conference*, pages 222–226, 2007. doi: 10.1109/CCNC.2007.51.
- [5] Jose Velazco. An inter planetary network enabled by smallsats. In *2020 IEEE Aerospace Conference*, pages 1–10, 2020. doi: 10.1109/AERO47225.2020.9172696.
- [6] Scott M. Johnson. A method for delivery of SMTP messages over Bundle Protocol networks. Internet-Draft draft-johnson-dtn-interplanetary-smtp-01, Internet Engineering Task Force, March 2025. URL <https://www.rfc-editor.org/drafts/draft-johnson-dtn-interplanetary-smtp-01>.

[//datatracker.ietf.org/doc/draft-johnson-dtn-interplanetary-smtp/01/](https://datatracker.ietf.org/doc/draft-johnson-dtn-interplanetary-smtp/01/).
Work in Progress.

- [7] Marc Blanchet. Encapsulation of Email over Delay-Tolerant Networks(DTN) using the Bundle Protocol. Internet-Draft draft-blanchet-dtn-email-over-bp-06, Internet Engineering Task Force, September 2025. URL <https://datatracker.ietf.org/doc/draft-blanchet-dtn-email-over-bp/06/>.
Work in Progress.
- [8] Lance Batac, Brian Contreras, Adrian Flores Aquino, Jiahao Li, Sophia Liwag, Nathan Luu, Nicholas Martin, Antonio Ortega Guerrero, Anderson Godo Pena Reyes, Ryan Torrez, and Jilei Zou. BPMail: Utilities for Transferring Email over Bundle Protocol. <https://github.com/250MHz/bpmail>, 2025. Accessed 2026-07-07.
- [9] Matthew Cosby and Wallace Tai. Lunar Communications Architecture Study Report. Technical report, Interagency Operations Advisory Group (IOAG), January 2022. URL <https://www.ioag.org/Public%20Documents/Lunar%20communications%20architecture%20study%20report%20FINAL%20v1.3.pdf>. Accessed 2026-07-07.
- [10] A. Pentland, R. Fletcher, and A. Hasson. Daknet: rethinking connectivity in developing nations. *Computer*, 37(1):78–83, 2004. doi: 10.1109/MC.2004.1260729.
- [11] S. Guo, M. H. Falaki, E. A. Oliver, S. Ur Rahman, A. Seth, M. A. Zaharia, and S. Keshav. Very low-cost internet access using kiosknet. *SIGCOMM Comput. Commun. Rev.*, 37(5):95–100, October 2007. ISSN 0146-4833. doi: 10.1145/1290168.1290181. URL <https://doi.org/10.1145/1290168.1290181>.
- [12] Shrikant Naidu, Suresh Chintada, Moushumi Sen, and Seshadri Raghavan. Challenges in deploying a delay tolerant network. pages 65–72, 09 2008. doi: 10.1145/1409985.1409998.

- [13] Scott M. Johnson. An Interplanetary DNS Model. Internet-Draft draft-johnson-dtn-interplanetary-dns-04, Internet Engineering Task Force, March 2025. URL <https://datatracker.ietf.org/doc/draft-johnson-dtn-interplanetary-dns/04/>. Work in Progress.
- [14] Emmanuel Gbenga Dada, Joseph Stephen Bassi, Haruna Chiroma, Shafi'i Muhammad Abdulhamid, Adebayo Olusola Adetunmbi, and Opeyemi Emmanuel Ajibuwa. Machine learning for email spam filtering: review, approaches and open research problems. *Heliyon*, 5(6):e01802, 2019. ISSN 2405-8440. doi: <https://doi.org/10.1016/j.heliyon.2019.e01802>. URL <https://www.sciencedirect.com/science/article/pii/S2405844018353404>.
- [15] Ernesto Damiani, Sabrina Vimercati, Stefano Paraboschi, and Pierangela Samarati. An open digest-based technique for spam detection. volume 2004, pages 559–564, 01 2004.
- [16] The Spamhaus Project. DNSBL Usage and Blocklist Documentation. <https://www.spamhaus.org/blocklists/>, . Accessed 2026-07-07.
- [17] Apache SpamAssassin Project. Mail::SpamAssassin::Conf — SpamAssassin Configuration File. https://spamassassin.apache.org/full/3.4.x/doc/Mail_SpamAssassin_Conf.html, . Accessed 2026-07-07.
- [18] Apache SpamAssassin Project. Mail::SpamAssassin::Plugin::URIDNSBL — Look Up URLs Against DNS Blocklists. https://spamassassin.apache.org/full/3.4.x/doc/Mail_SpamAssassin_Plugin_URIDNSBL.html, . Accessed 2026-07-07.
- [19] Scott Kitterman. Sender Policy Framework (SPF) for Authorizing Use of Domains in Email, Version 1. RFC 7208, April 2014. URL <https://www.rfc-editor.org/info/rfc7208>.

- [20] Murray Kucherawy, Dave Crocker, and Tony Hansen. DomainKeys Identified Mail (DKIM) Signatures. RFC 6376, September 2011. URL <https://www.rfc-editor.org/info/rfc6376>.
- [21] Murray Kucherawy and Elizabeth Zwicky. Domain-based Message Authentication, Reporting, and Conformance (DMARC). RFC 7489, March 2015. URL <https://www.rfc-editor.org/info/rfc7489>.
- [22] Vsevolod Stakhov. Rspamd: Fast, Free and Open-Source Spam Filtering System. <https://rspamd.com/>. Accessed 2026-07-24.
- [23] Vipul's Razor / Cloudmark. Vipul's Razor: Distributed, Collaborative Spam Detection and Filtering Network. <https://razor.sourceforge.net/>. Accessed 2026-07-07.
- [24] Stephen Farrell, Susan Symington, Howard Weiss, and Peter Lovell. Delay-Tolerant Networking Security Overview. Internet-Draft draft-irtf-dtnrg-sec-overview-06, Internet Engineering Task Force, March 2009. URL <https://datatracker.ietf.org/doc/draft-irtf-dtnrg-sec-overview/06/>. Work in Progress.
- [25] Sushant Jain, Kevin Fall, and Rabin Patra. Routing in a delay tolerant network. *SIGCOMM Comput. Commun. Rev.*, 34(4):145–158, August 2004. ISSN 0146-4833. doi: 10.1145/1030194.1015484. URL <https://doi.org/10.1145/1030194.1015484>.
- [26] Marc Blanchet. Encapsulation of HTTP over Delay-Tolerant Networks(DTN) using the Bundle Protocol. Internet-Draft draft-blanchet-dtn-http-over-bp-04, Internet Engineering Task Force, September 2025. URL <https://datatracker.ietf.org/doc/draft-blanchet-dtn-http-over-bp/04/>. Work in Progress.

- [27] The Spamhaus Project. QNAME Minimization and Spamhaus DNSBLs — an Explanation. <https://www.spamhaus.org/resource-hub/dnsbl/qname-minimization-and-spamhaus-dnsbls/>, . Accessed 2026-07-24.
- [28] Paul Shupak. Network Tests Latency. <https://cwiki.apache.org/confluence/spaces/SPAMASSASSIN/pages/119544790/NetworkTestsLatency>, 2006. Message to the SpamAssassin users mailing list, 14 January 2006; archived on the Apache SpamAssassin wiki. Accessed 2026-07-24.
- [29] Alireza Pour. Minimizing the time of spam mail detection by relocating filtering system to the sender mail server. *International Journal of Network Security Its Applications*, 4:53–62, 03 2012. doi: 10.5121/ijnsa.2012.4204.
- [30] Arch Linux. Unbound — archwiki. URL <https://wiki.archlinux.org/title/Unbound>. Accessed 2026-07-02.